1. Backend Web Server & API (app.py)

This module is the application's core, handling web requests and orchestrating data flow.

- **Sub-Module: Flask Application Core**
    - **Purpose**: To initialize and run the web server.
    - **Components**:
        - app = Flask(__name__): The central Flask application instance.
        - if __name__ == "__main__":: The main execution block that starts the server.
        - app.run(...): Configures and launches the development server, making the application accessible on the network.
- **Sub-Module: API Endpoint Routing**
    - **Purpose**: To define all the web addresses (URLs) that the frontend can call to get data or perform actions.
    - **Components**:
        - @app.route('/'): Serves the main index.html user interface.
        - @app.route('/api/dashboard-data'): Provides a comprehensive JSON object with aggregated data for the dashboard, including lane stats, totals, and alerts.
        - @app.route('/api/insights-data'): An endpoint that calls get_insights_data() to fetch and return historical performance data from the database for the "Insights" tab.
        - @app.route('/flash/<action>'): Handles hardware control, sending "on" or "off" commands to the ESP32-CAM's LED.
        - @app.route('/api/set_signal'): A critical endpoint that allows the AI control loop to post its decisions (which lane to make green and for how long), directly influencing the SIGNAL_STATE.
- **Sub-Module: Data Aggregation & State Management**
    - **Purpose**: To act as a single source of truth, ensuring data consistency across all API endpoints.
    - **Components**:
        - get_unified_traffic_data(): The most critical function in this sub-module. It reads the live vehicle counts from the REALTIME_DATA global variable and constructs a complete, consistent state of all lanes.
        - get_dashboard_data(): A consumer of get_unified_traffic_data(). It calculates high-level metrics like total_vehicles and avg_congestion and formats the data specifically for the dashboard UI.

        - REALTIME_DATA: A global Python dictionary that acts as a memory buffer, holding the latest vehicle counts as updated by the detection module.

- **Sub-Module: Background Thread Management**
    - **Purpose**: To run the AI control loop concurrently without blocking the web server.
    - **Components**:
        - if os.environ.get("WERKZEUG_RUN_MAIN") == "true":: A check to prevent the background thread from starting twice when Flask is in debug mode.
        - dqn_thread = threading.Thread(target=run_dqn_control_loop, daemon=True): The creation of the separate thread that will run the AI's decision-making process.
        - atexit.register(dqn_manager.save_agent_state): A safety hook that ensures the AI agent's learned progress is saved to a file when the application is shut down.

2. AI Vehicle Detection Module (modules/detection.py)

This module is responsible for "seeing" and counting vehicles in video frames.

- **Sub-Module: Model Initialization (VehicleDetector.__init__)**
    - **Purpose**: To load and prepare the AI model for use.
    - **Components**:
        - self.device = "cuda" if ...: Automatically detects if a GPU (CUDA) is available for faster processing, otherwise falls back to the CPU.
        - self.model = YOLO(model_path): Loads the pre-trained YOLOv8 object detection model from its file.
        - self.vehicle_class_ids: A filter to ensure the model only counts relevant objects (cars, trucks, buses, motorcycles) and ignores others.
        - self.confidence_threshold: A parameter to tune the model's sensitivity, preventing false detections from low-confidence guesses.
- **Sub-Module: Frame Processing & Annotation (detect_vehicles)**
    - **Purpose**: To take a raw image, find vehicles, and draw visual feedback on it.
    - **Components**:
        - **Image Decoding**: Converts the incoming JPEG byte stream into an OpenCV image format that can be processed.
        - **Frame Resizing**: Downscales the image before detection, a key optimization that significantly speeds up processing.

        - **Inference**: self.model(resized_frame, ...) runs the actual AI detection on the resized frame.
        - **Bounding Box Scaling**: A crucial fix that scales the detected bounding box coordinates from the small, resized image back to the original, full-sized image's dimensions.
        - **Annotation**: cv2.rectangle(...) and cv2.putText(...) draw the green boxes and labels onto the original frame.
        - **Image Encoding**: Converts the annotated OpenCV image back into JPEG bytes to be sent over the web.

3. Rule-Based Traffic Control Module (modules/dqn.py)

This is the "brain" of the system, which makes intelligent, rule-based decisions.

- **Sub-Module: Agent & Action Definition**
    - **Purpose**: To define the possible actions and the structure of the agent.
    - **Components**:
        - ACTION_TIME_OPTIONS: A list defining the possible green light durations (e.g., 8, 12, 16 seconds).
        - ACTIONS: A comprehensive list of every possible action the agent can take (e.g., "turn Lane 1 green for 8 seconds", "turn Lane 1 green for 12 seconds", etc.).
        - DDQNAgent: A class that, while named for DQN, currently acts as a placeholder. Its methods for learning (remember, learn) are present but do not perform any operations, making it compatible with the control loop's structure.
- **Sub-Module: Decision Logic (TrafficDQNManager.get_action)**
    - **Purpose**: To implement the core, rule-based algorithm for choosing the best action.
    - **Components**:
        - **Ambulance Priority Rule**: The first check is to see if any lane has an ambulance. If so, it immediately gets a green light for the maximum duration.
        - **Congestion Priority Rule**: If there are no ambulances, the agent finds the lane with the highest number of vehicles (np.argmax(current_counts)).
        - **Dynamic Green Time Logic**: Based on the number of vehicles in the most congested lane, it assigns a short, medium, or long green light duration.
        - **Action Selection**: Translates the chosen lane and green time into the corresponding action index from the ACTIONS list.

- **Sub-Module: Performance Tracking (remember_experience, get_and_reset_cycle_metrics)**
    - **Purpose**: To calculate and log performance metrics even without active learning.
    - **Components**:
        - _calculate_reward(): A function that calculates a "reward" score based on how many vehicles were cleared. This is used for logging and analysis, providing a measure of the system's efficiency per cycle.
        - remember_experience(): This method is called by the main loop to log the outcome of an action and its calculated reward.
        - get_and_reset_cycle_metrics(): Aggregates the rewards logged during a cycle and returns the average, which is then saved to the log files.

4. Video Streaming & Hardware Integration (app.py)

This module handles the acquisition and delivery of all video feeds.

- **Sub-Module: Live Camera Streaming**
    - **Purpose**: To connect to the ESP32-CAM and stream its video feed.
    - **Components**:
        - generate_frames_on_demand(): Streams the raw, unprocessed video from the ESP32. It uses requests.get(..., stream=True) to efficiently handle the live MJPEG stream.
        - generate_detection_frames(): Streams the AI-processed video. It fetches frames from the ESP32 and passes each one to vehicle_detector.detect_vehicles before yielding the annotated result.
- **Sub-Module: Pre-recorded Video File Streaming**
    - **Purpose**: To use local video files as camera sources, which is essential for development, testing, and creating repeatable simulations.
    - **Components**:
        - VIDEO_FILES: A dictionary mapping lane IDs to their corresponding .mp4 file paths.
        - generate_frames_from_file(): A generator function that opens a video file using cv2.VideoCapture, reads it frame by frame, performs vehicle detection on it, and streams the result. It is configured to loop the video indefinitely.
- **Sub-Module: Fallback & Robustness**
    - **Purpose**: To ensure the system remains operational even if a camera source fails.
    - **Components**:

        - **Webcam Fallback**:
          Both generate_frames_on_demand and generate_detection_frames cont ain try...except blocks. If the connection to the ESP32 fails, they automatically
          call generate_frames_from_webcam or generate_detection_frames_from _webcam to switch to the server's local webcam as a backup source.

5. Frontend User Interface (UI)

This is the visual layer of the application that the user interacts with.

- **Sub-Module: Page Structure & Layout (index.html, style.css)**
    - **Purpose**: To define the static structure and appearance of the dashboard.
    - **Components**:
        - **Header & Tabs**: The main navigation structure.
        - **Camera Grid**: The layout for the four live video feeds.
        - **Component Cards**: The layout for the "AI Manageable Components" tab, organizing statistics, controls, and alerts.
        - **Insights Grid**: The layout for the charts on the "Insights" tab.
        - **Styling**: CSS files (style.css, components.css) define colors, fonts, spacing, and responsive design for different screen sizes.
- **Sub-Module: Client-Side Dynamic Updates (script.js)**
    - **Purpose**: To make the dashboard live and interactive by fetching and displaying data from the backend.
    - **Components**:
        - **Tab Switching Logic**: Handles showing and hiding the correct content when a user clicks a tab.
        - **Main Data Fetch Loop**: setInterval(updateDashboardData, 3000) is the heart of the UI. Every 3 seconds, it calls the /api/dashboard-data endpoint.
        - **DOM Manipulation**: After fetching data, the script updates dozens of elements on the page (e.g., vehicle counts, status indicators, distribution bars) by changing their text content and CSS classes.
        - updateTrafficControlCenter(): A dedicated function that updates the signal light colors and countdown timers on the "AI Manageable Components" tab based on the signal_state from the API.
- **Sub-Module: Data Visualization (insights.js, Chart.js)**
    - **Purpose**: To render historical data into interactive charts.
    - **Components**:

        - fetchAndRenderInsights(): A function in script.js that is called when the "Insights" tab is opened. It fetches data from the /api/insights-data endpoint.
        - initializeCharts() (in insights.js): Takes the fetched data and uses the Chart.js library to create and render the four performance charts (Average Vehicles, Action Reasons, etc.).
        - **Chart Canvases** (index.html): The <canvas> elements that act as containers for the charts.

6. Data Logging and Analytics Module

This module is responsible for recording historical data for analysis and persistence.

- **Sub-Module: CSV & SQLite Initialization (app.py)**
    - **Purpose**: To prepare the logging mechanisms when the application starts.
    - **Components**:
        - initialize_log_file(): Checks if traffic_log.csv exists and, if not, creates it and writes the header row.
        - initialize_database(): Connects to the traffic_log.db SQLite file and executes a CREATE TABLE IF NOT EXISTS command to ensure the traffic_logs table is ready.
- **Sub-Module: Data Persistence (app.py)**
    - **Purpose**: To write a new record of system state and AI decisions at the end of each control loop cycle.
    - **Components**:
        - append_to_log(): Opens the CSV file in append mode and writes a new row.
        - append_to_db(): Connects to the SQLite database and executes a parameterized INSERT statement to add a new record, preventing SQL injection.
- **Sub-Module: Data Analysis & Retrieval (modules/insights.py)**
    - **Purpose**: To query the database, process the raw data, and calculate meaningful statistics.
    - **Components**:
        - get_db_connection(): A utility function to connect to the SQLite database.
        - get_insights_data(): The main analysis function. It uses the powerful pandas library to read the entire traffic_logs table into a DataFrame. It then performs aggregation tasks like calculating means (.mean()), counting values (.value_counts()), and grouping data

(.groupby()) to compute the final statistics required by the frontend charts.